

Summary: DAX Functions

SESSION OVERVIEW:

By the end of this session, students will be able to:

- Understand why we use DAX
- Understand fundamental concepts, and basic operators in DAX
- Understand various functions and various types of calculations in DAX
- Learn the concept of the calculated date table

KEY TOPICS AND EXAMPLES:

1. Introduction to DAX

DAX stands for **Data Analysis Expressions**. It is a formula language used in Power BI and Power Pivot in Excel. DAX comprises a variety of functions, operators, and constants, allowing for the formulation of expressions to compute and deliver single or multiple values.

a. Why do we use DAX?

- DAX enables us to derive insights easily from large datasets.
- DAX helps us solve real world problems through functionalities like measures, and calculated columns.
- DAX helps us get the most out of our data.

b. Fundamental concepts of DAX

- Syntax:** It is how the formula is written: the grammar and the structure. Understanding syntax ensures your formulas are written correctly and can be interpreted by Power BI. For example, to find the sum, you need to write =SUM().
- Functions:** They are formulas that use arguments to calculate and find results. For example, mathematical functions, date/time functions and logical functions etc.
- Context:** DAX formulas operate within a specific context. It refers to the current selection or filter applied to the data when the formula is evaluated. For example, we may want to calculate the total sales in India, but based on the 'Customer Region' filter, we may just end up calculating total sales in North India.

There are two types of context in Power BI:

- **Row context:** This refers to the specific row of data that the formula is currently working on. When you create a calculated column or measure, DAX considers the values in each individual row.

For example: A formula calculating profit might reference the "Sales" and "Cost" columns within the same row to determine the profit for that specific product or customer.

- **Filter context:** This context is established by filters applied to your data through slicers, drill-downs, or other user interactions within the report. The formula only considers data that meets the current filter criteria.

For example: While calculating total sales for a specific product category, if we filter the report for a particular sales region, the formula will only consider sales figures for that product category within the selected region.

c. DAX Operators

Let's delve deeper and explore how DAX operators can manipulate data, compare values, and unlock hidden insights in your reports. We will see this through an example of a general 'sales' dataset.

i. Scenario: Cracking the Sales Code:

Imagine you're analyzing a massive dataset of your company's sales figures. You want to identify regions exceeding their sales targets and those lagging behind. Here's where the DAX operator comes in:

- **Comparison Operators:** Use operators like ' $>$ ' (greater than) and ' $<$ ' (less than) to compare actual sales figures with target values. Build a calculated column that flags regions exceeding targets (Actual Sales $>$ Target Sales).
- **Logical Operators:** Combine these comparisons with $\&\&$ (AND) and $\parallel$ (OR) to create more complex filters. For example, identify regions exceeding targets AND with a growth rate higher than 10% "(Actual Sales $>$ Target Sales) $\&\&$ (Percentage Change in Sales $>$ 10%)".

Another example would be to segment customers by purchase frequency - Identify customers who have purchased more than twice this year: "(Number of Purchases This Year $>$ 2)".

ii. Unveiling Customer Trends:

Understanding customer behavior is crucial. DAX operators can help you analyze trends:

- **Text Concatenation Operator:** Combine text elements with the $\&$ (ampersand) operator. It creates a new column that combines a customer's first and last name for better readability in reports.
- **Arithmetic Operators:** They are used to perform basic arithmetic functions like addition, subtraction, etc. For example, if we want to add Sales in Jan, Sales in Feb, and Sales in March, we can simply use the ' $+$ ' operator to add these 3 columns and create a 4th column for the Sales in the quarter (Jan - March).

The most common types of operators used in DAX are:

- Arithmetic Operators ($+$, $-$, $*$, $/$)
- Comparison Operator ($=$, $>$, $<$, $>=$, $<=$, $<>$)

- Logical Operator (&&, ||, IN)
- Text Concatenation Operator (&)

Analytic Query: It is a DAX query statement that retrieves data from a dataset and produces a result from a data model. Analytic queries in DAX typically involve functions like CALCULATE, FILTER, SUMMARIZE, and others to define the logic for data retrieval and analysis.

2. DAX Calculations

Like Excel formulas, DAX formulas also start with '=' sign. DAX has three kinds of calculations:

a. Calculated Tables

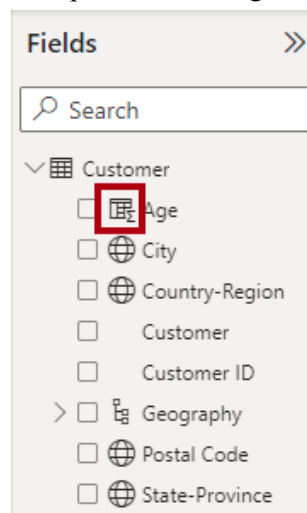
Calculated tables are derived tables created using the current model data. They increase the model storage size. The best example of a calculated table is the Date Table, which is explained later in this session.

b. Calculated Columns

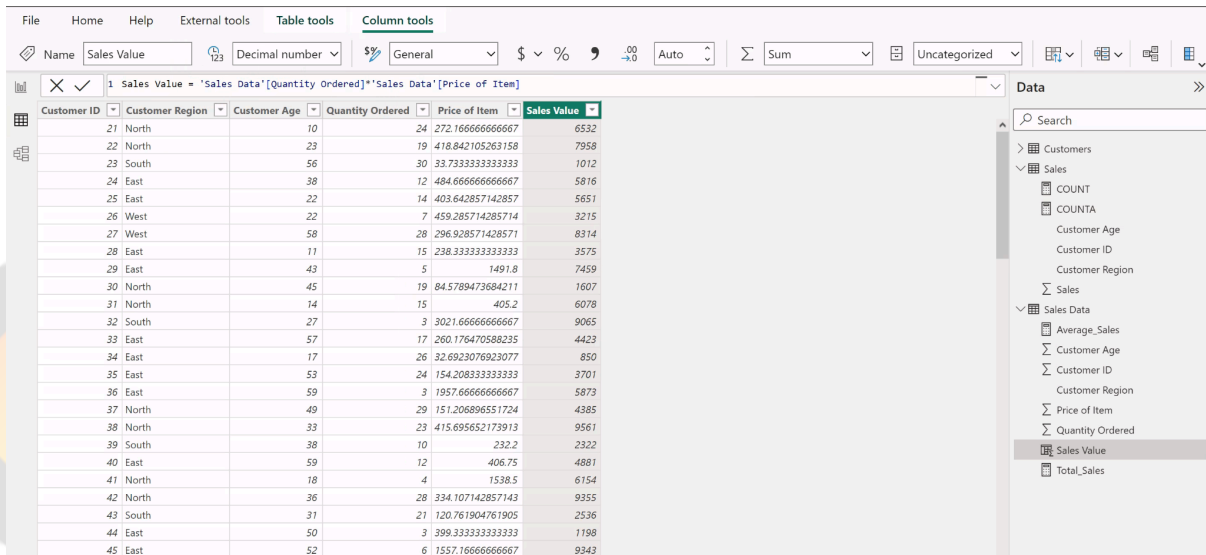
We can create a new column using DAX based on another column. They are used when we want to calculate a value for every row in the table. They are used to filter/slice the data. Calculated columns occupy space in the memory and are stored in the data model

The **Custom column** option in the **Add Column** tab in Power Query Editor or the **'New Column'** option in the **Modelling** tab in Power BI Desktop.

In the **Fields pane**, calculated columns are shown with a special icon. The following example shows a single calculated column in the **Customer table** called 'Age'.



For example, in [this dataset](#), we can add a calculated column - 'Sales Value', which is the **product** of the 'Price of Item' and the 'Quantity Ordered'.



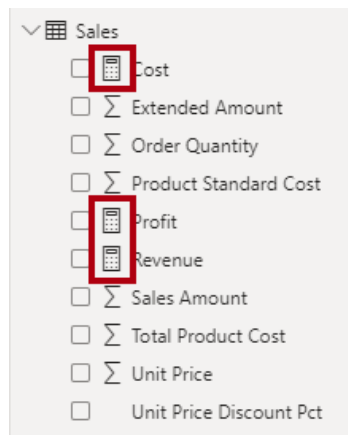
Customer ID	Customer Region	Customer Age	Quantity Ordered	Price of Item	Sales Value
21	North	10	24	272.166666666667	6532
22	North	23	19	418.842105263158	7958
23	South	56	30	33.7333333333333	1012
24	East	38	12	484.666666666667	5816
25	East	22	14	403.642857142857	5651
26	West	22	7	459.285714285714	3215
27	West	58	28	296.928571428571	8314
28	East	11	15	238.333333333333	3575
29	East	43	5	1491.8	7459
30	North	45	19	84.5789473684211	1607
31	North	14	15	405.2	6078
32	South	27	3	3021.66666666667	9065
33	East	57	17	260.176470588235	4423
34	East	17	26	32.6923076923077	850
35	East	53	24	154.208333333333	3701
36	East	59	3	1957.66666666667	5873
37	North	49	29	151.206896551724	4385
38	North	33	23	415.695652173913	9561
39	South	38	10	232.2	2322
40	East	59	12	406.75	4881
41	North	18	4	1538.5	6154
42	North	36	28	334.107142857143	9355
43	South	31	21	120.761904761905	2536
44	East	50	3	399.333333333333	1198
45	East	52	6	1557.16666666667	9343

c. Measures

Measures are calculations used to summarize data. Measures are evaluated at query time, unlike calculated columns, which are calculated at data refresh time. Measures do not occupy space in the model as their results are not stored in the data model.

Examples of Measures: sum, average, count, min, and max.

In the **Fields pane**, measures are shown with the **calculator** icon. The following example shows three measures in the **Sales** table: Cost, Profit, and Revenue.



To create a new measure, click on the 'Sales' table → go to the 'Table tools' tab in the ribbon → select Quick measure/new measure and start typing the formula.

Alternatively, right click on the table in the field list of Report View and click on **New Measure** option.

Quick Measure: They are defined using pre-defined formulas in the system.

New Measure: They are defined by the user.

Commonly asked question: What is the difference between measure and calculated column?

Answer : The main difference is that measures are aggregations of the data that **change with context** - as filters/slicers/'row selection'/'column selection' are applied, the value of the measure changes. They are usually used where we want to display calculations reflecting user selections.

Calculated columns on the other hand are computed during a data refresh and do not depend on user selections. They are static and **the values do not change with context**.

3. DAX Logical Functions

a. OR

The OR function returns TRUE if at least one argument evaluates to TRUE; otherwise, it returns FALSE.

Syntax: OR(<logical1>, <logical2>, ...)

b. AND

The AND function returns TRUE if both arguments evaluate to TRUE; otherwise, it returns FALSE.

Syntax: AND(<logical1>, <logical2>, ...)

c. NOT

The NOT function returns TRUE if the argument evaluates to FALSE, and FALSE if the argument evaluates to TRUE.

Syntax: NOT(<logical>)

d. IF

The IF function evaluates a condition and returns one value if the condition is true and another value if the condition is false.

Syntax: IF(<logical_test>, <value_if_true>, <value_if_false>)

Example 1: In our dataset, if we want to classify customers into High and Low Sales Value categories, we can use the following conditions: Price of Item is > 500 and Quantity Ordered >= 20.

IF('Sales Data'[Price of Item] > 500 && 'Sales Data'[Quantity Ordered] >= 20, "High Sales Value", "Low Sales Value")

Example 2: From our dataset, for an advertisement campaign we do not want to target people who are either below 20 years of age or whose region is East or West, then we can use the conditions below.

IF('Sales Data'[Customer Age] < 20 || 'Sales Data'[Customer Region] IN {"East", "West"}, "Not in Target Segment", "In Target Segment")

Nested IF:

It is used when we have multiple conditions and we want to evaluate them sequentially, we use nested IF. We can return different results based on multiple criteria using Nested IFs.

```
IF(<condition1>, <value_if_true1>,
    IF(<condition2>, <value_if_true2>,
        IF(<condition3>, <value_if_true3>,
            <value_if_false>
        )
    )
)
```

e. IFERROR

The IFERROR function returns the result of the expression if it does not generate an error; otherwise, it returns the specified value.

Syntax: IFERROR(<expression>, <value_if_error>)

f. ISEEMPTY

The ISEEMPTY function returns TRUE if the expression evaluates to an empty value (NULL); otherwise, it returns FALSE.

Syntax: ISEEMPTY(<expression>)

g. ISBLANK

The ISBLANK function returns TRUE if the expression evaluates to BLANK; otherwise, it returns FALSE.

Syntax: ISBLANK(<expression>)

h. SWITCH

The SWITCH function evaluates an expression against a list of values and returns a result corresponding to the first matching value. If no match is found, it returns the else_result. It is like the Nested IF statement.

Syntax: SWITCH(<expression>, <value1>, <result1>, <value2>, <result2>, ..., <else_result>)

For example: In our dataset, we can use the Sales Value column and define three categories of sales: High, Medium and Low.

If Sales Value ≤ 3000 , the category will be “Low Sales”

If Sales Value > 3000 and Sales Value ≤ 5000 , the category will be “Medium Sales”

If Sales Value > 5000 , the category will be “High Sales”.

For defining this, we can use the SWITCH function as shown below.

File Home Help External tools Table tools Measure tools

Name Sales Category Home table Sales Data Format Text \$ % , .00 Auto Data category Uncategorized

```

1 Sales Category = SWITCH(TRUE(),
2 SUM('Sales Data'[Sales Value]) <= 3000, "Low Sales",
3 AND(SUM('Sales Data'[Sales Value]) > 3000 , SUM('Sales Data'[Sales Value]) <= 5000), "Medium Sales",
4 SUM('Sales Data'[Sales Value]) > 5000, "High Sales" )
5
6

```

Customer ID	Customer Region	Customer Age	Quantity Ordered	Price of Item	Sales Value
21	North	10	24	272.166666666667	6532
22	North	23	19	418.842105263158	7958
23	South	56	30	33.7333333333333	1012
24	East	38	12	484.666666666667	5816
25	East	22	14	403.642857142857	5651
26	West	22	7	459.285714285714	3215
27	West	58	28	296.928571428571	8314
28	East	11	15	238.333333333333	3575
29	East	43	5	1491.8	7459
30	North	45	19	84.5789473684211	1607
31	North	14	15	405.2	6078
32	South	27	3	3021.66666666667	9065
33	South	57	17	260.176470588235	4423
34	East	17	26	32.6923076923077	850
35	East	53	24	154.208333333333	3701
36	East	59	3	1957.66666666667	5873
37	North	49	29	151.206896551724	4385
38	North	33	23	415.695652173913	9561

Data

- Customers
- Sales
- Sales Data
 - Average_Sales
 - Customer Age
 - Customer ID
 - Customer Region
 - Price of Item
 - Quantity Ordered
 - Sales Category
 - Sales Value
 - Total_Sales

This is the Table view. If we want to see the actual values in this measure, we will have to see that in the report view, as shown below.

Report View

File Home Insert Modeling View Optimize Help External tools

Get data New visual More visuals New measure Sensitivity Publish

Filters

Visualizations

Data

Table View

Low Sales

Medium Sales

High Sales

Column

Calculated Column

Sales Value	Sales Category
2,740.00	Low Sales
2,600.00	Low Sales
2,945.00	Low Sales
3,215.00	Medium Sales
3,373.00	Medium Sales
3,425.00	Medium Sales
3,575.00	Medium Sales
3,690.00	Medium Sales
3,701.00	Medium Sales
3,858.00	Medium Sales
3,892.00	Medium Sales
3,971.00	Medium Sales
4,128.00	Medium Sales
4,137.00	Medium Sales
4,285.00	Medium Sales
4,385.00	Medium Sales
4,392.00	Medium Sales
4,423.00	Medium Sales
4,639.00	Medium Sales
4,642.00	Medium Sales
4,881.00	Medium Sales
4,912.00	Medium Sales
4,928.00	Medium Sales
4,970.00	Medium Sales
5,135.00	High Sales
5,434.00	High Sales
Total	High Sales

In the table shown above, the calculated column Sales value is shown along with the measure: Sales Category. This measure is created using the SWITCH statement.

4. DAX Aggregate Functions

Note: For practicing DAX in this session, we will be using [this dataset](#).

a. MIN

It returns the minimum value (numerical) in a column.

Syntax: =MIN ([Column])

For Example: In our dataset, if we want to create a new column which stores the **minimum price** if "Price of Item is less than 200", **minimum quantity** if "Price of Item is between 200 and 400" and **minimum total sales** if "Price of Item is more than 400", we can use Nested IF:


```

IF('Sales Data'[Price of Item] < 200,
MIN('Sales Data'[Price of Item]),
IF('Sales Data'[Price of Item] < 400,
MIN('Sales Data'[Quantity Ordered]),
MIN('Sales Data'[Total Sales])
)
)

```

b. MAX

It returns the maximum value (numerical) in a column.

Syntax: =MAX ([Column])

c. SUM

It returns the sum of all the numbers in the column'

Syntax: =SUM ([Column])

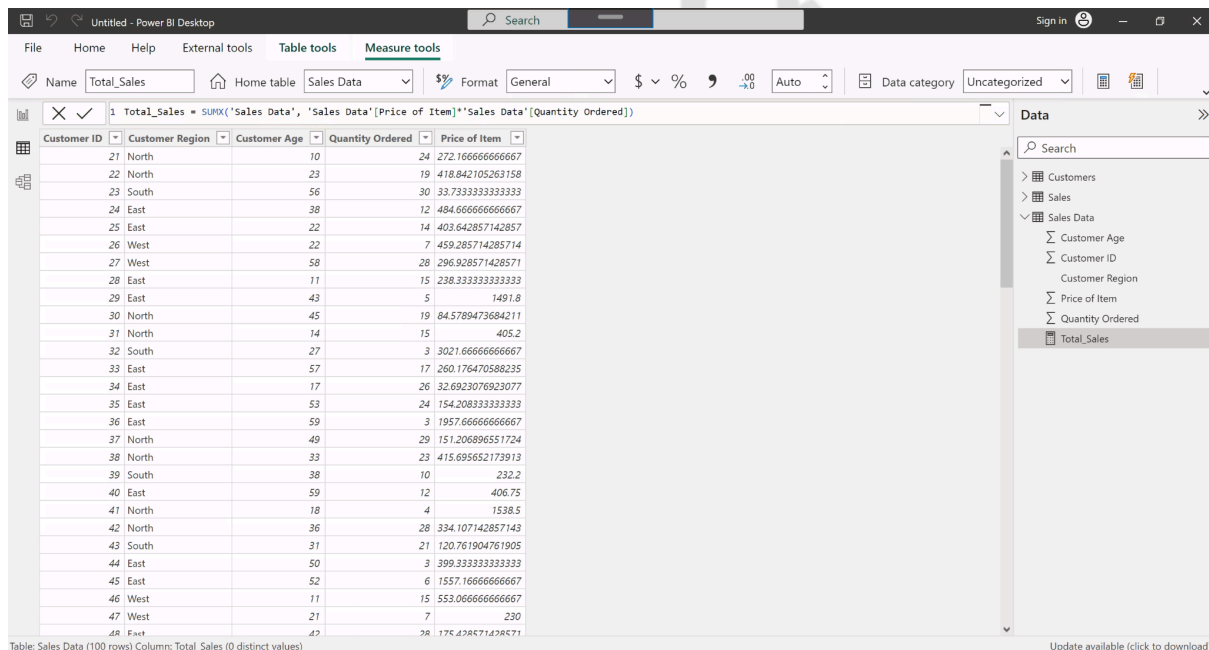
d. SUMX

It is an **iterative** function which returns the sum of an expression evaluated for each row in a table. It iterates on each row, performs the expression and does the sum on the result of the previous iteration.

Syntax: =SUMX(Table, Expression)

For example: In the [sales dataset](#), we want the total sum of sales. We can use a measure for this, using the SUMX aggregation. This formula can be used:

Total_Sales = SUMX('Sales Data','Sales Data'[Price of Item]'Sales Data'[Quantity Ordered])*



Customer ID	Customer Region	Customer Age	Quantity Ordered	Price of Item	Total_Sales
21	North	10	24	272.166666666667	
22	North	23	19	418.842105263158	
23	South	56	30	33.7333333333333	
24	East	38	12	484.666666666667	
25	East	22	14	403.642857142857	
26	West	22	7	459.285714285714	
27	West	58	28	296.928571428571	
28	East	11	15	238.333333333333	
29	East	43	5	1491.8	
30	North	45	19	84.5789473684211	
31	North	14	15	405.2	
32	South	27	3	3021.66666666667	
33	East	57	17	260.176470588235	
34	East	17	26	32.6923076923077	
35	East	53	24	154.208333333333	
36	East	59	3	1957.66666666667	
37	North	49	29	151.206896551724	
38	North	33	23	415.695652173913	
39	South	38	10	232.2	
40	East	59	12	406.75	
41	North	18	4	1538.5	
42	North	36	28	334.107142857143	
43	South	31	21	120.761904761905	
44	East	50	3	399.333333333333	
45	East	52	6	1557.16666666667	
46	West	11	15	553.066666666667	
47	West	21	7	230	
48	East	47	28	179.428571428571	

Note: If you want to calculate running total within the current month, then the TOTALMTD function is more useful than the SUMX function.

e. AVERAGE

It returns the average of the values in a column

Syntax: =AVERAGE ([Column])

For Example: From our Dataset, we can demonstrate this function using a scenario. For example, To increase sales, our company has adopted a new policy that reduces the price of expensive items. The policy says that all items with price above 1000 will now take the average price and all items with price below 1000 will take the actual current price. In that case, we can use this formula in DAX using IF and AVERAGE statements.

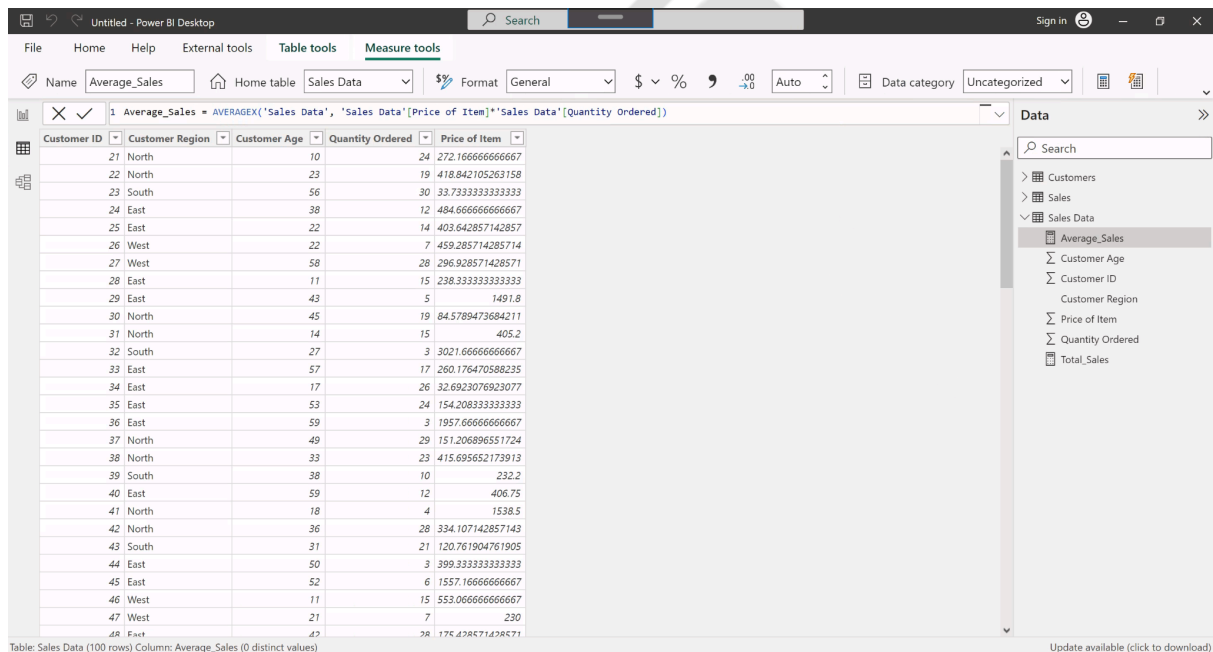
Syntax: IF('Sales Data'[Price of Item]>1000, AVERAGE('Sales Data'[Price of Item]), 'Sales Data'[Price of Item])

f. AVERAGEX

It is an **iterative** function which returns the average of an expression evaluated for each row in a table. It iterates on each row, performs the expression, takes the resulting set of values and calculates the arithmetic mean of those values.

Syntax: =AVERAGEX(Table, Expression)

For example, if we want to find the Average of the sale for each customer ID, we can AVERAGEX to first calculate the Sale in each row and find the average of each of those sales values. This formula can be used: *Average_Sales = AVERAGEX('Sales Data', 'Sales Data'[Price of Item]*'Sales Data'[Quantity Ordered])*



Customer ID	Customer Region	Customer Age	Quantity Ordered	Price of Item
21	North	10	24	272.166666666667
22	North	23	19	418.842105263158
23	South	56	30	73.7333333333333
24	East	38	12	484.666666666667
25	East	22	14	403.642857142857
26	West	22	7	459.285714285714
27	West	58	28	296.928571428571
28	East	11	15	238.333333333333
29	East	43	5	1491.8
30	North	45	19	84.5789473684211
31	North	14	15	405.2
32	South	27	3	3021.66666666667
33	East	57	17	260.176470588235
34	East	17	26	32.6923076923077
35	East	53	24	154.208333333333
36	East	59	3	1957.66666666667
37	North	49	29	151.206896551724
38	North	33	23	415.695652173913
39	South	38	10	232.2
40	East	59	12	406.75
41	North	18	4	1538.5
42	North	36	28	334.107142857143
43	South	31	21	120.761904761905
44	East	50	3	399.333333333333
45	East	52	6	1557.16666666667
46	West	11	15	553.066666666667
47	West	21	7	230
48	East	47	38	175.428571428571

5. DAX Count Functions

a. DISTINCTCOUNT

It returns the distinct count of values in a column. If we have multiple instances of the same value it counts it only once.

Syntax: =DISTINCTCOUNT([Column])

b. COUNT

It returns the count of numerical values in a column. If we have multiple instances of the same value it counts each of them separately.

Syntax: =COUNT([Column])

Note: COUNTX allows us to perform the count with a filter context.

c. COUNTA

It counts the number of records in the column that are not blank. It is different from COUNT in that it counts the non-numerical values also.

Syntax: =COUNTA([Column])

d. COUNTROWS

It counts the number of rows in a table.

Syntax: =COUNTROWS([Column])

e. COUNTBLANK

It counts the number of blank cells in a column.

Syntax: =COUNTBLANK([Column])

6. DAX Mathematical Functions

a. ABS

It finds the absolute value of the specified number.

Syntax: ABS(<number>)

b. SQRT

It finds the Square Root of the specified number.

Syntax: SQRT(<number>)

c. POWER

It raises the number to the specified power.

Syntax: POWER(<number>, <power>)

d. RAND

It returns a random number between 0 and 1.

Syntax: RAND()

e. EXP

It returns the exponential value of a number (e^x).

Syntax: EXP(<number>)

f. LOG

It returns the logarithm with the base of the specified number.

Syntax: LOG(<number>, <base>)

7. DAX Date Table

CALENDAR Function in DAX

It creates a calendar table containing a contiguous range of dates between the start and end dates.

Syntax: CALENDAR(<start_date>, <end_date>)

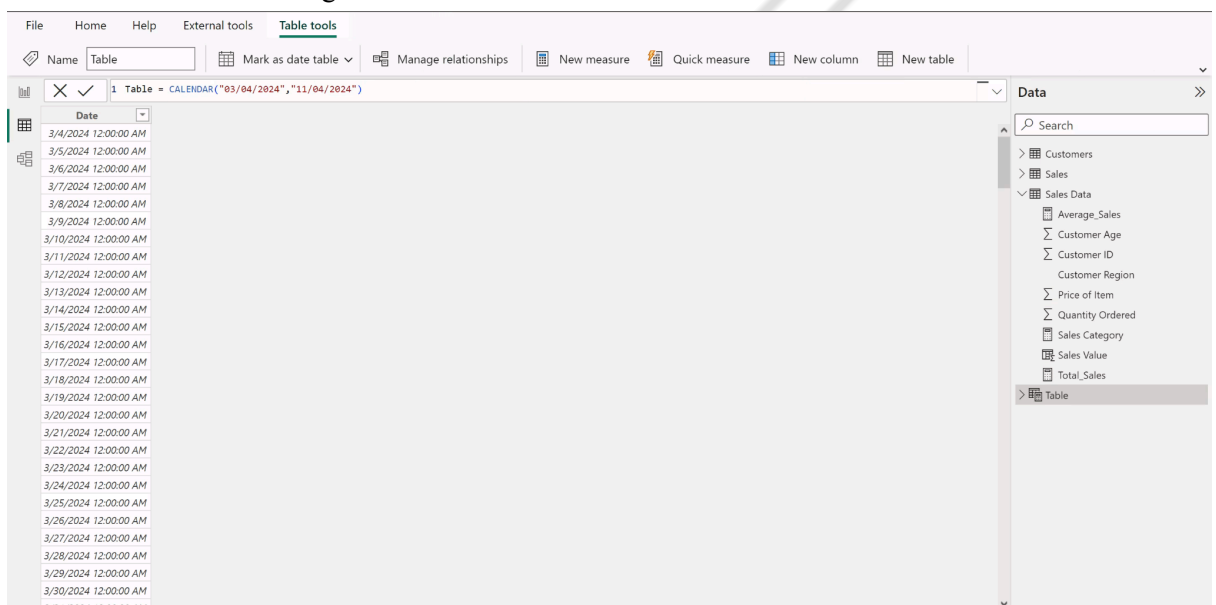
Date Table

This calculated table is called a **Date Table** and is very useful in Power BI.

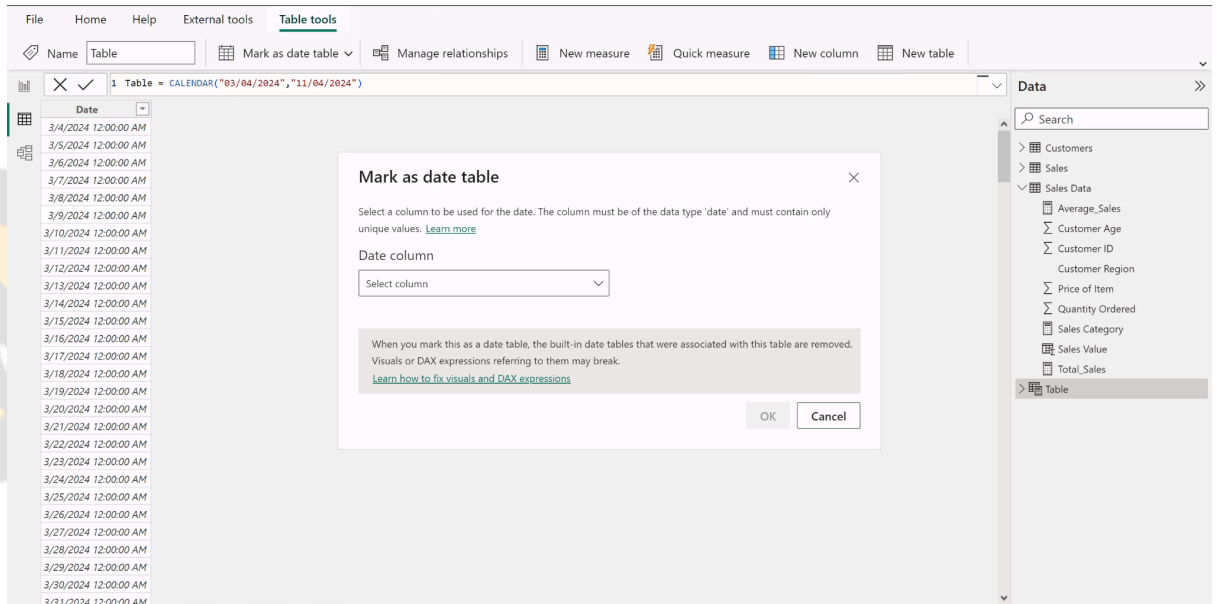
Let's say we have an Order Date (date on which item was ordered) and a Delivery Date (date on which item was delivered) in our dataset. We want to add a filter/slicer for dates in our Power BI report.

If we are filtering data on the Order date, it will not be consistent with the Delivery Date. For example if we want just orders placed and delivered in the first week of March, we will have to use two separate filters since the date ranges are different for both the date columns. The best way to tackle such a scenario is to use a central date table and to use relationships (refer: Data Modelling concept taught in previous class). Filtering would become easier in this manner.

This can be seen in the image below:

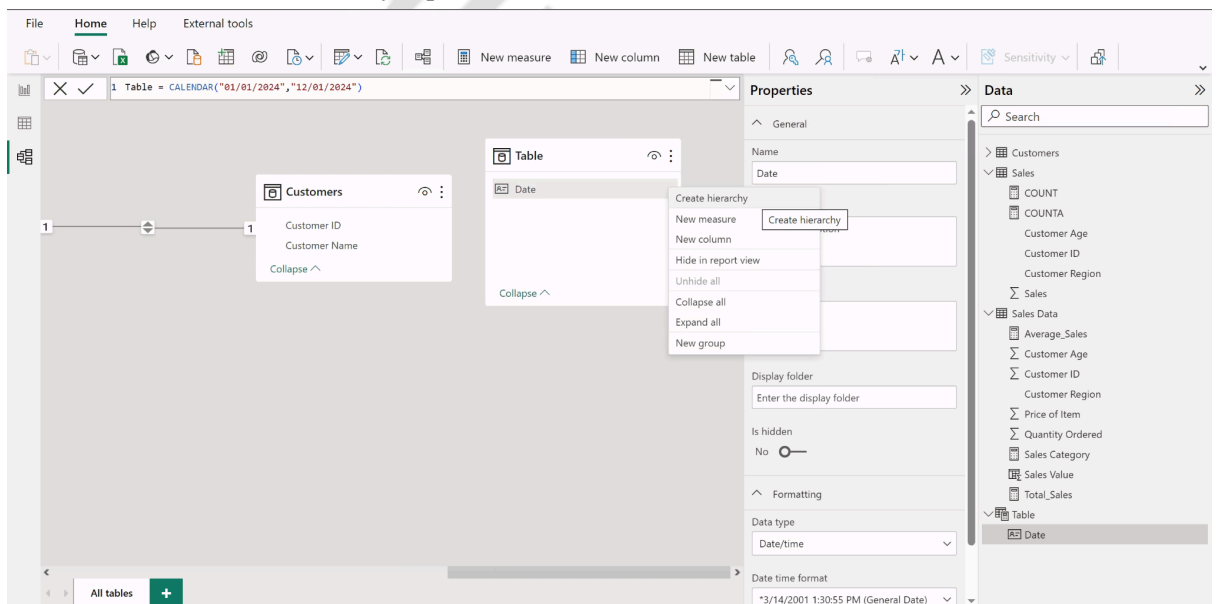


On the ribbon there is an option to “Mark as date table”. On selecting this option, it shows a dialog box to select the column that we want to use for date in that table.



On selecting the date column, click OK.

You can define a hierarchy in the Date table: year, quarter, month, day etc. this makes filtering very easy. To create a hierarchy, go to the Model view, right-click the Date column in the data table and select ‘Create hierarchy’ option.



8. DAX Text Functions

a. CONCATENATE

It combines multiple text strings into a single text string.

Syntax: CONCATENATE(<text1>, <text2>, ...)

b. LEFT

It returns the leftmost characters from a text string, up to the specified number of characters.

Syntax: *LEFT*(<text>, <num_chars>)

c. RIGHT

It returns the rightmost characters from a text string, up to the specified number of characters.

Syntax: *RIGHT*(<text>, <num_chars>)

d. MID

It returns a substring from a specified text string. The substring starts from the specified position and extends for the specified number of characters.

Syntax: *MID*(<text>, <start_num>, <num_chars>)

e. LEN

It returns the length of a string in terms of the number of characters in the string.

Syntax: *LEN*(<text>)

f. TRIM

It removes leading and trailing spaces from a text string.

Syntax: *TRIM*(<text>)

g. UPPER

It converts all characters in a text string to uppercase.

Syntax: *UPPER*(<text>)

h. LOWER

It converts all characters in a text string to lowercase.

Syntax: *LOWER*(<text>)

i. REPLACE

It replaces part of a text string with another text string, based on the specified starting position and length.

Syntax: *REPLACE*(<text>, <start>, <num_chars>, <new_text>)

j. TEXT

It converts a value to text using the specified format.

Syntax: *TEXT*(<value>, <text_format>)